```
import numpy as np
import pandas as pd


import matplotlib.pyplot as plt
import seaborn as sns


import warnings
warnings.filterwarnings("ignore")


from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler


from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier,
GradientBoostingClassifier


from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)


import joblib


plt.style.use("ggplot")
sns.set_theme()

print("All libraries imported successfully.")
```

```
All libraries imported successfully.
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
import pandas as pd
import os

DATA_PATH = "/content/drive/MyDrive/StockData"
```

```
import os

print(os.listdir(DATA_PATH))
```

```
['all_companies_classification_data.csv',
'merged_stock_sentiment_data.csv', 'stock_data_raw.csv',
'stock_preprocessed.csv', 'stock_cleaned.csv']
```

```
import os

print("Current Directory:")
print(os.getcwd())

print("\nFiles/Folders:")
for item in os.listdir():
    print(item)
```

```
Current Directory:
/content

Files/Folders:
.config
drive
sample_data
```

```
import pandas as pd

df = pd.read_csv("merged_stock_sentiment_data.csv")

print(df.shape)
df.head()
```

```
(12069, 14)
```


```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 12069,\n  \"fields\":
[\n    {\n      \"column\": \"Adj Close\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 81.72035639199635,\n
\"min\": 1.9600000381469729,\n        \"max\": 409.9700012207031,\n
\"num_unique_values\": 5001,\n        \"samples\": [\n
82.05149841308594,\n          166.30650329589844,\n
225.1666717529297\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Close\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 81.42248087872433,\n        \"min\": 1.9600000381469729,\n
\"max\": 409.9700012207031,\n        \"num_unique_values\": 4882,\n
\"samples\": [\n          182.0200042724609,\n
23.059499740600582,\n          224.22999572753903\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"High\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 83.28601485510593,\n
\"min\": 2.006666898727417,\n        \"max\": 414.4966735839844,\n
```

```
\"num_unique_values\": 4839,\n        \"samples\": [\n
144.24349975585938,\n          157.81900024414062,\n
189.97999572753903\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Low\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 79.47151690176024,\n        \"min\": 1.909999966621399,\n
\"max\": 405.6666564941406,\n        \"num_unique_values\": 4861,\n
\"samples\": [\n          66.91533660888672,\n
242.3999938964844,\n          153.79299926757812\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Open\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 81.42505424801288,\n
\"min\": 2.0,\n        \"max\": 411.4700012207031,\n
\"num_unique_values\": 4826,\n        \"samples\": [\n
15.28933334350586,\n          50.212501525878906,\n
24.83333206176757\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Volume\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 101819062,\n        \"min\": 19444500,\n        \"max\":
1506120000,\n        \"num_unique_values\": 5054,\n
\"samples\": [\n          60442000,\n          322344000,\n
359934400\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Company\",\n      \"properties\": {\n        \"dtype\":
\"category\",\n        \"num_unique_values\": 3,\n        \"samples\":
[\n          \"Apple\",\n          \"Amazon\",\n          \"Tesla\"\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"Date\",\n      \"properties\":
{\n        \"dtype\": \"object\",\n        \"num_unique_values\":
2487,\n        \"samples\": [\n          \"2021-08-13\",\n
\"2021-03-04\",\n          \"2019-04-17\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Target\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\": 0,\n
\"min\": -1,\n        \"max\": 1,\n        \"num_unique_values\": 3,\n
\"samples\": [\n          1,\n          -1,\n          0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Score\",\n      \"properties\": {\
n        \"dtype\": \"number\",\n        \"std\": 3863,\n
\"min\": 0,\n        \"max\": 108617,\n        \"num_unique_values\":
2575,\n        \"samples\": [\n          2815,\n          2686,\n
186\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Comments\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 2162,\n        \"min\": 0,\n
\"max\": 90822,\n        \"num_unique_values\": 1379,\n
\"samples\": [\n          1246,\n          732,\n          810\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"Cleaned_Text\",\n
```

\"properties\": {\n   \"dtype\": \"string\",\n
\"num_unique_values\": 11492,\n   \"samples\": [\n
\"last 3 quarter apple tanked 15 post earnings long trading position
planning selling close earnings making ath right anticipate little run
earnings week however dont see blowout earnings quarter expect another
pullback another monster run q4 edit im talking midlong term trading
position long term investment\",\n   \"thats 30 today reach
3250 next two week premium roughly 7080\",\n   \"hi team im
working timeline fire wondering way run phased montecarlo simulation
mean right expense still really high mortgage child going school
however school expense drop house paid plus well renting half inlaws
live expense drop even im looking way run montecarlo simulation least
3 phase different expense level advice run calculation excel website
allows multiphase approach x200b edit based everyones input created
basic spreadsheet run multiphase simulation find obviously changed
initial amount logic phase 1 1 7 year retire still planning adding
savingsinvestment amount b6 2 calculating random return building
previous year column f g 3 take end amount 7th year run 1000 iteration
4 calculate mean median standard deviation d2e4 5 calculate percentile
1 5 10 d5e8 phase 2 1 take median phase 1 2 instead adding
savingsinvestment amount taking certain amount every year i6 3 repeat
step 25 phase 1 x200b repeat many phase needed im wondering taking
median right choice mean would better choice x200b thank advice
feedback\"\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"Sentiment\",\n   \"properties\": {\n   \"dtype\":
\"category\",\n   \"num_unique_values\": 3,\n   \"samples\":
[\n   \"Positive\",\n   \"Neutral\",\n
\"Negative\"\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"Sentiment_Score\",\n   \"properties\": {\n   \"dtype\":
\"number\",\n   \"std\": 0.10104073125484946,\n   \"min\":
0.37898767,\n   \"max\": 1.0,\n   \"num_unique_values\":
8036,\n   \"samples\": [\n   0.99974555,\n
0.9978811,\n   0.9987282\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   }\n]\n}","type":"dataframe","variable_name":"df"}

```
print("=" * 60)
print("DATASET INFORMATION")
print("=" * 60)

# Shape
print(f"Rows    : {df.shape[0]}")
print(f"Columns : {df.shape[1]}")

print("\n" + "=" * 60)
print("COLUMN NAMES")
print("=" * 60)
```

```python
print(df.columns.tolist())

print("\n" + "=" * 60)
print("DATA TYPES")
print("=" * 60)

print(df.dtypes)

print("\n" + "=" * 60)
print("MISSING VALUES")
print("=" * 60)

print(df.isnull().sum())

print("\n" + "=" * 60)
print("DUPLICATE ROWS")
print("=" * 60)

print(df.duplicated().sum())

print("\n" + "=" * 60)
print("STATISTICAL SUMMARY")
print("=" * 60)

display(df.describe(include='all'))
```

```
============================================================
DATASET INFORMATION
============================================================
Rows    : 12069
Columns : 14

============================================================
COLUMN NAMES
============================================================
['Adj Close', 'Close', 'High', 'Low', 'Open', 'Volume', 'Company',
'Date', 'Target', 'Score', 'Comments', 'Cleaned_Text', 'Sentiment',
'Sentiment_Score']

============================================================
DATA TYPES
============================================================
Adj Close          float64
Close              float64
High               float64
Low                float64
Open               float64
Volume               int64
Company             object
Date                object
```

```
Target                int64
Score                 int64
Comments              int64
Cleaned_Text         object
Sentiment            object
Sentiment_Score     float64
dtype: object

============================================================
MISSING VALUES
============================================================
Adj Close             0
Close                 0
High                  0
Low                   0
Open                  0
Volume                0
Company               0
Date                  0
Target                0
Score                 0
Comments              0
Cleaned_Text        154
Sentiment             0
Sentiment_Score       0
dtype: int64

============================================================
DUPLICATE ROWS
============================================================
0

============================================================
STATISTICAL SUMMARY
============================================================

{"summary":"{\n  \"name\": \"display(df\",\n  \"rows\": 11,\n
\"fields\": [\n    {\n      \"column\": \"Adj Close\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
4215.134976485707,\n        \"min\": 1.9600000381469729,\n
\"max\": 12069.0,\n        \"num_unique_values\": 8,\n
\"samples\": [\n          143.9076746364884,\n
149.892333984375,\n          12069.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Close\",\n      \"properties\": {\
n        \"dtype\": \"number\",\n        \"std\": 4215.034145727699,\n
\"min\": 1.9600000381469729,\n        \"max\": 12069.0,\n
\"num_unique_values\": 8,\n        \"samples\": [\n
144.54327995011244,\n          150.82000732421875,\n          12069.0\
n        ],\n        \"semantic_type\": \"\",\n
```

```
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"High\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 4214.339383155522,\n        \"min\": 2.006666898727417,\n
\"max\": 12069.0,\n        \"num_unique_values\": 8,\n
\"samples\": [\n          147.05109821676427,\n
153.1699981689453,\n          12069.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Low\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 4215.776141954225,\n
\"min\": 1.909999966621399,\n        \"max\": 12069.0,\n
\"num_unique_values\": 8,\n        \"samples\": [\n
141.89498841665744,\n          148.49000549316406,\n          12069.0\
n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Open\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 4214.980603489735,\n        \"min\": 2.0,\n        \"max\":
12069.0,\n        \"num_unique_values\": 8,\n        \"samples\": [\n
144.54118416202485,\n          150.63999938964844,\n          12069.0\
n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Volume\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 507954182.18619055,\n        \"min\": 12069.0,\n
\"max\": 1506120000.0,\n        \"num_unique_values\": 8,\n
\"samples\": [\n          118224484.94490016,\n          90696000.0,\n
12069.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Company\",\n      \"properties\": {\n        \"dtype\":
\"category\",\n        \"num_unique_values\": 4,\n        \"samples\":
[\n          3,\n          \"4322\",\n          \"12069\"\n        ],\
n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"Date\",\n      \"properties\":
{\n        \"dtype\": \"date\",\n        \"min\": \"1970-01-01
00:00:00.000000025\",\n        \"max\": \"2024-06-17 00:00:00\",\n
\"num_unique_values\": 4,\n        \"samples\": [\n          2487,\n
\"25\",\n          \"12069\"\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"Target\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 4267.039849169909,\n        \"min\": -
1.0,\n        \"max\": 12069.0,\n        \"num_unique_values\": 5,\n
\"samples\": [\n          -0.07399121716795094,\n          1.0,\n
0.9972171189644964\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Score\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 37740.41108123087,\n        \"min\": 0.0,\n        \"max\":
108617.0,\n        \"num_unique_values\": 8,\n        \"samples\": [\n
1015.0982682906621,\n          61.0,\n          12069.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Comments\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\":
```

```
31624.87610495384,\n        \"min\": 0.0,\n        \"max\": 90822.0,\n
\"num_unique_values\": 8,\n        \"samples\": [\n
415.17267379236057,\n          60.0,\n          12069.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Cleaned_Text\",\n
\"properties\": {\n        \"dtype\": \"category\",\n
\"num_unique_values\": 4,\n        \"samples\": [\n          11492,\n
\"139\",\n          \"11915\"\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"Sentiment\",\n      \"properties\": {\n
\"dtype\": \"category\",\n        \"num_unique_values\": 4,\n
\"samples\": [\n          3,\n          \"9396\",\n
\"12069\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Sentiment_Score\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 4266.761779788833,\n        \"min\":
0.10104073125484946,\n        \"max\": 12069.0,\n
\"num_unique_values\": 8,\n        \"samples\": [\n
0.9594485562631535,\n          0.99949336,\n          12069.0\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    }\n  ]\n}","type":"dataframe"}
```


```
print(df.columns.tolist())
```

```
['Adj Close', 'Close', 'High', 'Low', 'Open', 'Volume', 'Company',
'Date', 'Target', 'Score', 'Comments', 'Cleaned_Text', 'Sentiment',
'Sentiment_Score']
```

```
print("=" * 60)
print("MISSING VALUES")
print("=" * 60)

print(df.isnull().sum())

print("\nDuplicate Rows :", df.duplicated().sum())

df.drop_duplicates(inplace=True)


df["Date"] = pd.to_datetime(df["Date"])

print("\nDataset Shape After Cleaning :", df.shape)
```

```
============================================================
MISSING VALUES
============================================================
Adj Close            0
Close                0
High                 0
Low                  0
Open                 0
```

```
Volume                0
Company               0
Date                  0
Target                0
Score                 0
Comments              0
Cleaned_Text        154
Sentiment             0
Sentiment_Score       0
dtype: int64

Duplicate Rows : 0

Dataset Shape After Cleaning : (12069, 14)
```

```
print("First 5 Rows")
display(df.head())

print("\nLast 5 Rows")
display(df.tail())

print("\nRandom Sample")
display(df.sample(5))

print("\nDataset Info")
df.info()
```

```
First 5 Rows
```


```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 5,\n  \"fields\": [\n{\n      \"column\": \"Adj Close\",\n      \"properties\": {\n\"dtype\": \"number\",\n        \"std\": 1.2225576291792017,\n\"min\": 8.532785415649414,\n        \"max\": 11.446333885192873,\n\"num_unique_values\": 5,\n        \"samples\": [\n8.712499618530273,\n          11.446333885192873,\n8.778499603271484\n        ],\n        \"semantic_type\": \"\",\n\"description\": \"\"\n      }\n    },\n    {\n      \"column\":\"Close\",\n      \"properties\": {\n        \"dtype\": \"number\",\n\"std\": 2.0700055601486556,\n        \"min\": 8.712499618530273,\n\"max\": 13.569286346435549,\n        \"num_unique_values\": 5,\n\"samples\": [\n          8.712499618530273,\n13.569286346435549,\n          8.778499603271484\n        ],\n\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"High\",\n      \"properties\": {\n\"dtype\": \"number\",\n        \"std\": 2.017272107834298,\n\"min\": 8.897000312805176,\n        \"max\": 13.602856636047363,\n\"num_unique_values\": 5,\n        \"samples\": [\n8.897000312805176,\n          13.602856636047363,\n8.949999809265137\n        ],\n        \"semantic_type\": \"\",\n\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
```

```
\"Low\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 1.9850817275699641,\n        \"min\": 8.679499626159668,\n
\"max\": 13.282142639160156,\n        \"num_unique_values\": 5,\n
\"samples\": [\n          8.68649959564209,\n
13.282142639160156,\n          8.679499626159668\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Open\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 1.9667645033945806,\n
\"min\": 8.685999870300293,\n        \"max\": 13.321429252624512,\n
\"num_unique_values\": 5,\n        \"samples\": [\n
8.816499710083008,\n          13.321429252624512,\n
8.685999870300293\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Volume\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 263980348,\n        \"min\": 84050000,\n        \"max\":
658677600,\n        \"num_unique_values\": 5,\n        \"samples\": [\
n          84050000,\n          467832400,\n          116210000\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"Company\",\n
\"properties\": {\n        \"dtype\": \"category\",\n
\"num_unique_values\": 2,\n        \"samples\": [\n
\"Amazon\",\n          \"Apple\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Date\",\n      \"properties\": {\n
\"dtype\": \"date\",\n        \"min\": \"2010-09-20 00:00:00\",\n
\"max\": \"2011-09-12 00:00:00\",\n        \"num_unique_values\": 5,\n
\"samples\": [\n          \"2010-12-13 00:00:00\",\n          \"2011-
09-12 00:00:00\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Target\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 0,\n        \"min\": -1,\n        \"max\": 1,\n
\"num_unique_values\": 2,\n        \"samples\": [\n          -1,\n
1\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Score\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 3,\n        \"min\": 0,\n        \"max\": 8,\n
\"num_unique_values\": 4,\n        \"samples\": [\n          7,\n
5\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Comments\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 6,\n        \"min\": 0,\n
\"max\": 16,\n        \"num_unique_values\": 4,\n        \"samples\":
[\n          5,\n          16\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"Cleaned_Text\",\n      \"properties\": {\n
\"dtype\": \"string\",\n        \"num_unique_values\": 5,\n
\"samples\": [\n          \"hussman warning awful time invest\",\n
\"personally im fan theyre already beatendown price im well aware
terrible fundamentalsfinancials im hoping price kick sensationalism
```

```
basis least sound\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Sentiment\",\n      \"properties\": {\n        \"dtype\":
\"string\",\n        \"num_unique_values\": 3,\n        \"samples\":
[\n          \"Positive\",\n          \"Neutral\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Sentiment_Score\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.19472371472917738,\n        \"min\": 0.5534017,\n        \"max\":
0.99989474,\n        \"num_unique_values\": 5,\n        \"samples\":
[\n          0.5534017,\n          0.99398685\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    }\n  ]\n}","type":"dataframe"}
```


Last 5 Rows


```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 5,\n  \"fields\": [\n
{\n      \"column\": \"Adj Close\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 16.93278463443272,\n
\"min\": 205.7400054931641,\n        \"max\": 237.3300018310547,\n
\"num_unique_values\": 3,\n        \"samples\": [\n
205.7400054931641,\n          207.88999938964844,\n
237.3300018310547\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Close\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 16.93278463443272,\n        \"min\": 205.7400054931641,\n
\"max\": 237.3300018310547,\n        \"num_unique_values\": 3,\n
\"samples\": [\n          205.7400054931641,\n
207.88999938964844,\n          237.3300018310547\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"High\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 16.4241386758605,\n
\"min\": 207.63999938964844,\n        \"max\": 237.8099975585937,\n
\"num_unique_values\": 3,\n        \"samples\": [\n
207.63999938964844,\n          208.1999969482422,\n
237.8099975585937\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Low\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 15.925228040045416,\n        \"min\": 204.58999633789065,\n
\"max\": 233.97000122070312,\n        \"num_unique_values\": 3,\n
\"samples\": [\n          205.0500030517578,\n
204.58999633789065,\n          233.97000122070312\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Open\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 15.460209061209817,\n
\"min\": 205.8300018310547,\n        \"max\": 234.8099975585937,\n
\"num_unique_values\": 3,\n        \"samples\": [\n
206.97999572753903,\n          205.8300018310547,\n
234.8099975585937\n        ],\n        \"semantic_type\": \"\",\n
```

\"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"Volume\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 1525687,\n   \"min\": 24892400,\n   \"max\": 28481400,\n   \"num_unique_values\": 3,\n   \"samples\": [\n   28061600,\n   24892400,\n   28481400\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"Company\",\n   \"properties\": {\n   \"dtype\": \"category\",\n   \"num_unique_values\": 2,\n   \"samples\": [\n   \"Apple\",\n   \"Amazon\"\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"Date\",\n   \"properties\": {\n   \"dtype\": \"date\",\n   \"min\": \"2024-11-27 00:00:00\",\n   \"max\": \"2024-11-29 00:00:00\",\n   \"num_unique_values\": 2,\n   \"samples\": [\n   \"2024-11-29 00:00:00\",\n   \"2024-11-27 00:00:00\"\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"Target\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 0,\n   \"min\": 0,\n   \"max\": 1,\n   \"num_unique_values\": 2,\n   \"samples\": [\n   0,\n   1\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"Score\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 13,\n   \"min\": 0,\n   \"max\": 31,\n   \"num_unique_values\": 2,\n   \"samples\": [\n   31,\n   0\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"Comments\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 131,\n   \"min\": 2,\n   \"max\": 307,\n   \"num_unique_values\": 4,\n   \"samples\": [\n   307,\n   2\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"Cleaned_Text\",\n   \"properties\": {\n   \"dtype\": \"string\",\n   \"num_unique_values\": 5,\n   \"samples\": [\n   \"please use thread discussion dont feel warrant new post sub rule posting question basic personal financeinvesting topic relaxed little bit rule memesspamselfpromotionexcessive rudenesspolitics still apply look faq subreddit posting see question frequently asked since post tend get busy consider sorting comment new instead best top see newest post\",\n   \"disabled mobility impairment brain impairment 2 year ago came across phone leasing plan great camera great automated assistance sweet deal time time upgrade mother need phone save money 2 u iam going buy one give saved money due increased productivity ability take really photo video fly work film also film school saved time though professional movie camera cut set time walkthroughs seeing filmed scene would look like updated sku phone come 2 version 63 68 inch actually 67 measurement current phone 67 though like look feel smaller phone worried smaller screen going decrease productivity ability type

```
get thing done end hindrance help right im using voice assist type im worried might overthinking price difference two 100 u im already spending 100 extra memory upgrade phone thatd 200 beyond price part black friday deal looked id paying either 15 month smaller one 20 month bigger one theyre phone different size might overthinking sickly person really relies ease use getting gaming hot pizzazz stuff strictly productivity mother wasnt phoneless id probably hold phone another year could save money family thats im going know use cuz shes pretty sickly well thanks angelica\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Sentiment\",\n
\"properties\": {\n        \"dtype\": \"category\",\n
\"num_unique_values\": 1,\n        \"samples\": [\n
\"Positive\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Sentiment_Score\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.002370541791789363,\n        \"min\":
0.9944608,\n        \"max\": 0.99998724,\n
\"num_unique_values\": 5,\n        \"samples\": [\n
0.99991035\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\n}","type":"dataframe"}
```


Random Sample


```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 5,\n  \"fields\": [\n
{\n      \"column\": \"Adj Close\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 57.25557233564281,\n
\"min\": 92.6082992553711,\n        \"max\": 241.02999877929688,\n
\"num_unique_values\": 5,\n        \"samples\": [\n
184.47000122070312,\n          241.02999877929688,\n
181.7100067138672\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Close\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 56.48798030922686,\n        \"min\": 95.04000091552734,\n
\"max\": 241.02999877929688,\n        \"num_unique_values\": 5,\n
\"samples\": [\n          184.47000122070312,\n
241.02999877929688,\n          181.7100067138672\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"High\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 66.92700195872722,\n
\"min\": 95.2300033569336,\n        \"max\": 271.0,\n
\"num_unique_values\": 5,\n        \"samples\": [\n
186.77999877929688,\n          271.0,\n          185.8600006103516\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"Low\",\n      \"properties\": {\
n        \"dtype\": \"number\",\n        \"std\": 56.17977128712881,\n
\"min\": 93.7125015258789,\n        \"max\": 239.6499938964844,\n
\"num_unique_values\": 5,\n        \"samples\": [\n
180.5800018310547,\n          239.6499938964844,\n
```

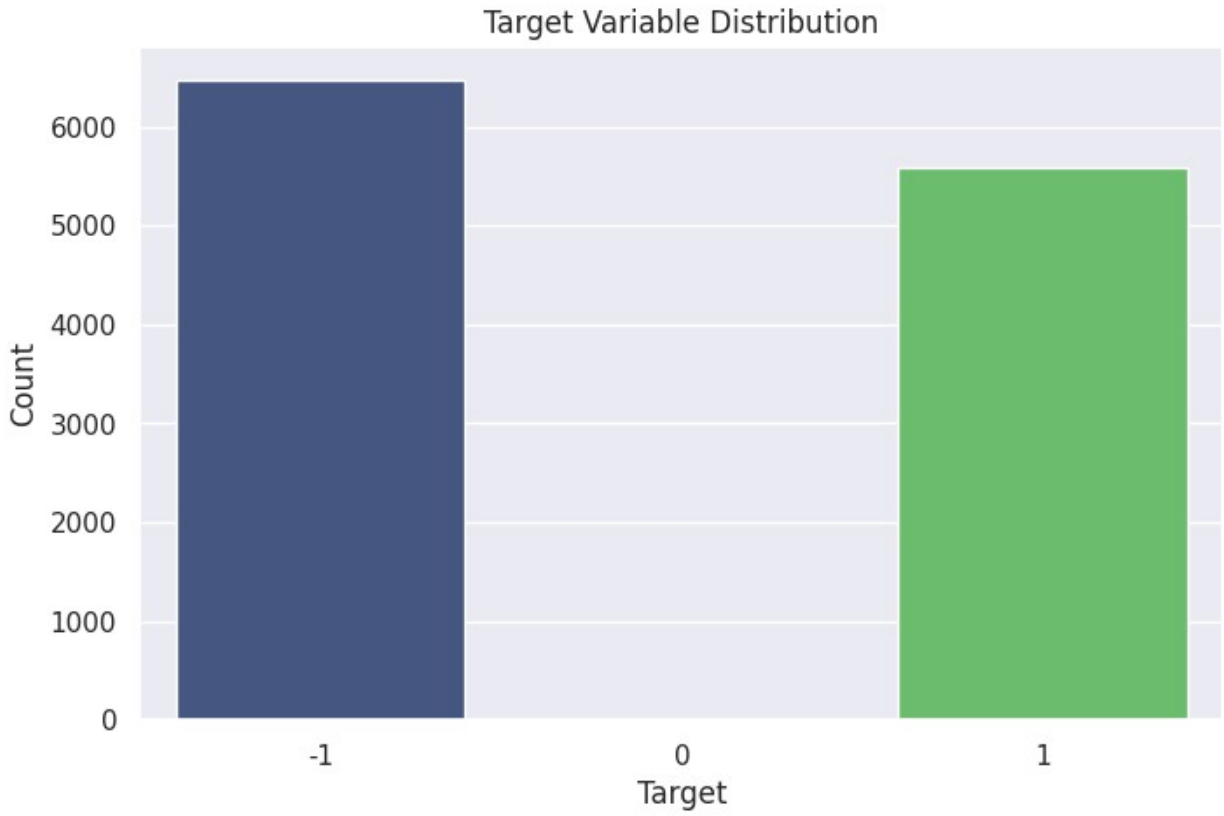
```
179.3800048828125\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Open\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 64.56309217943388,\n        \"min\": 93.75,\n        \"max\":
263.29998779296875,\n        \"num_unique_values\": 5,\n
\"samples\": [\n          186.5399932861328,\n
263.29998779296875,\n          184.72000122070312\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Volume\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\": 70721954,\n
\"min\": 39508600,\n        \"max\": 221707300,\n
\"num_unique_values\": 5,\n        \"samples\": [\n
96870700,\n          221707300,\n          39508600\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Company\",\n      \"properties\":
{\n        \"dtype\": \"string\",\n        \"num_unique_values\": 3,\n
\"samples\": [\n          \"Amazon\",\n          \"Tesla\",\n
\"Apple\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Date\",\n      \"properties\": {\n        \"dtype\": \"date\",\n
\"min\": \"2020-07-29 00:00:00\",\n        \"max\": \"2024-07-30
00:00:00\",\n        \"num_unique_values\": 5,\n        \"samples\":
[\n          \"2023-05-25 00:00:00\",\n          \"2024-07-11
00:00:00\",\n          \"2024-07-30 00:00:00\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Target\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\": 1,\n
\"min\": -1,\n        \"max\": 1,\n        \"num_unique_values\": 2,\n
\"samples\": [\n          -1,\n          1\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Score\",\n      \"properties\": {\
n        \"dtype\": \"number\",\n        \"std\": 740,\n
\"min\": 1,\n        \"max\": 1683,\n        \"num_unique_values\":
5,\n        \"samples\": [\n          1214,\n          1683\
n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Comments\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 313,\n        \"min\": 5,\n
\"max\": 671,\n        \"num_unique_values\": 5,\n        \"samples\":
[\n          671,\n          489\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Cleaned_Text\",\n
\"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 5,\n        \"samples\": [\n
\"automaker dip come competitor ford motor jumped nine spot land 32
meanwhile broadly respondent survey chose patagonia costco us 1 2
reputable brand looked negatively upon another one musk company giving
twitter ranking 97 three spot survey laggard trump organization\",\n
\"good day gay bear\"\n        ],\n        \"semantic_type\": \"\",\n
```

\"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"Sentiment\",\n   \"properties\": {\n   \"dtype\": \"string\",\n   \"num_unique_values\": 3,\n   \"samples\": [\n   \"Positive\",\n   \"Negative\"\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"Sentiment_Score\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 0.0666834876592832,\n   \"min\": 0.86528146,\n   \"max\": 0.9996388,\n   \"num_unique_values\": 5,\n   \"samples\": [\n   0.86528146,\n   0.89258003\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   }\n]\n}","type":"dataframe"}

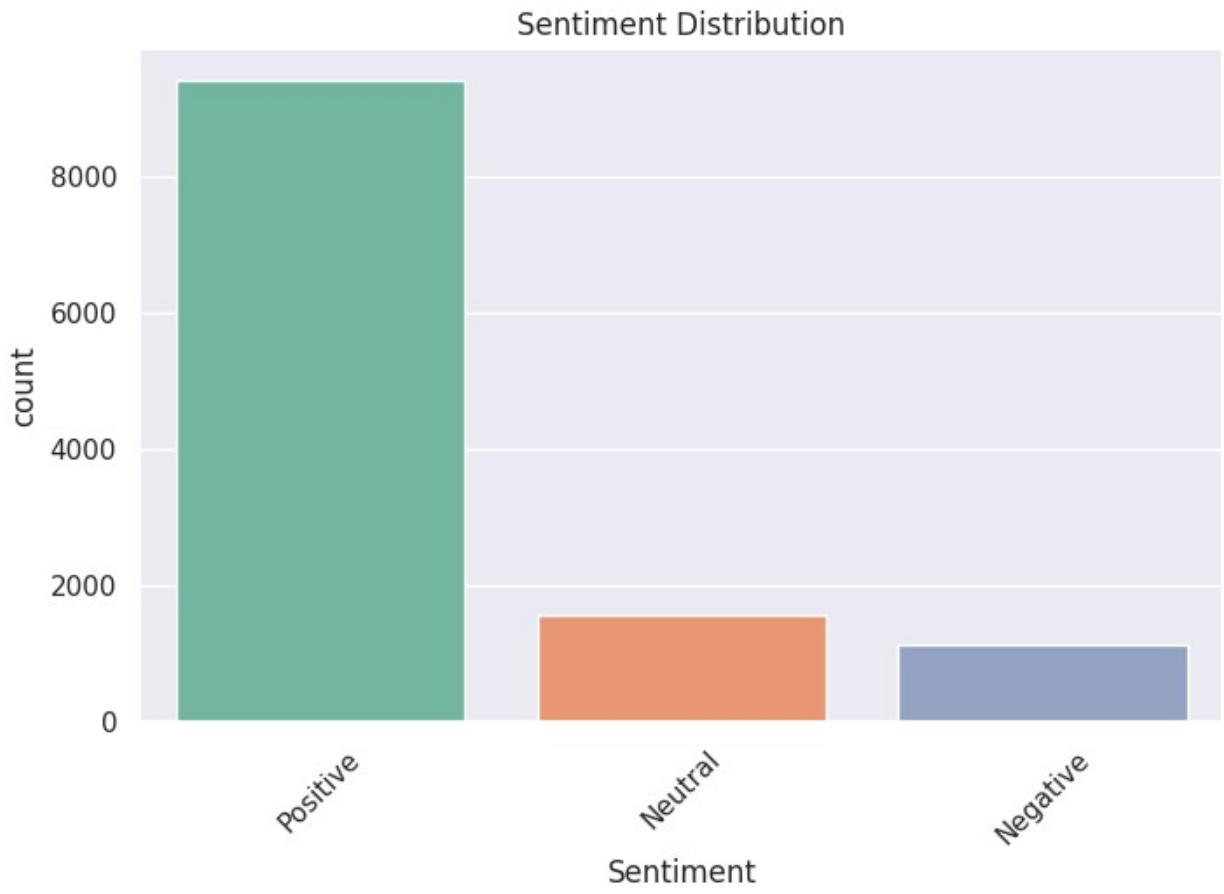
```
Dataset Info
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 12069 entries, 0 to 12068
Data columns (total 14 columns):
 #   Column           Non-Null Count  Dtype         
---  ------           --------------  -----         
 0   Adj Close        12069 non-null  float64       
 1   Close            12069 non-null  float64       
 2   High             12069 non-null  float64       
 3   Low              12069 non-null  float64       
 4   Open             12069 non-null  float64       
 5   Volume           12069 non-null  int64         
 6   Company          12069 non-null  object        
 7   Date             12069 non-null  datetime64[ns]
 8   Target           12069 non-null  int64         
 9   Score            12069 non-null  int64         
 10  Comments         12069 non-null  int64         
 11  Cleaned_Text     11915 non-null  object        
 12  Sentiment        12069 non-null  object        
 13  Sentiment_Score  12069 non-null  float64       
dtypes: datetime64[ns](1), float64(6), int64(4), object(3)
memory usage: 1.3+ MB
```

```python
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(8,5))

sns.countplot(
    x="Target",
    data=df,
    palette="viridis"
)

plt.title("Target Variable Distribution")
plt.xlabel("Target")
```

```
plt.ylabel("Count")

plt.show()
```



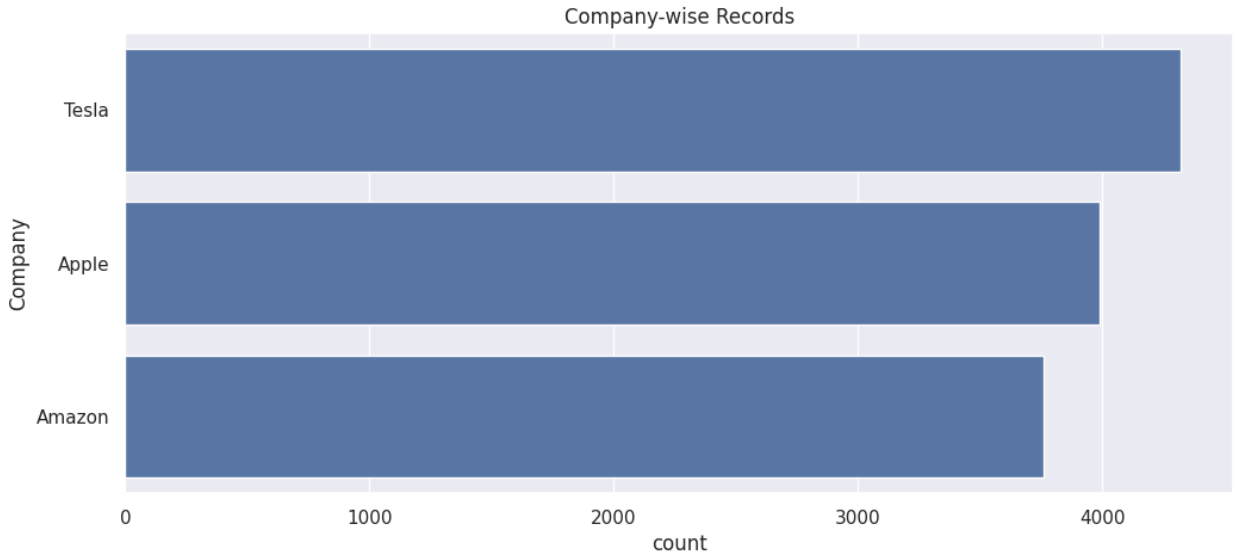

```
plt.figure(figsize=(8,5))

sns.countplot(
    x="Sentiment",
    data=df,
    palette="Set2"
)

plt.title("Sentiment Distribution")

plt.xticks(rotation=45)

plt.show()
```




```
print(df["Target"].unique())
```

```
[ 1 -1  0]
```

```
plt.figure(figsize=(12,5))

sns.countplot(
    y="Company",
    data=df,
    order=df["Company"].value_counts().index
)

plt.title("Company-wise Records")

plt.show()
```




```
features = [
    "Open",
    "High",
    "Low",
    "Close",
    "Adj Close",
    "Volume",
    "Score",
    "Sentiment_Score"
]

X = df[features]

# Target
y = df["Target"]

print("Feature Shape :", X.shape)
print("Target Shape  :", y.shape)

X.head()
```

```
Feature Shape : (12069, 8)
Target Shape  : (12069,)
```


```
{"summary":"{\n  \"name\": \"X\",\n  \"rows\": 12069,\n  \"fields\":
[\n    {\n      \"column\": \"Open\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 81.42505424801288,\n
\"min\": 2.0,\n        \"max\": 411.4700012207031,\n
\"num_unique_values\": 4826,\n        \"samples\": [\n
15.28933334350586,\n          50.212501525878906,\n
24.83333206176757\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"High\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
```

\"std\": 83.28601485510593,\n   \"min\": 2.006666898727417,\n
\"max\": 414.4966735839844,\n   \"num_unique_values\": 4839,\n
\"samples\": [\n   144.24349975585938,\n
157.81900024414062,\n   189.97999572753903\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   },\n   {\n   \"column\": \"Low\",\n   \"properties\": {\n
\"dtype\": \"number\",\n   \"std\": 79.47151690176024,\n
\"min\": 1.909999966621399,\n   \"max\": 405.6666564941406,\n
\"num_unique_values\": 4861,\n   \"samples\": [\n
66.91533660888672,\n   242.3999938964844,\n
153.79299926757812\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"Close\",\n   \"properties\": {\n   \"dtype\": \"number\",\n
\"std\": 81.42248087872433,\n   \"min\": 1.9600000381469729,\n
\"max\": 409.9700012207031,\n   \"num_unique_values\": 4882,\n
\"samples\": [\n   182.0200042724609,\n
23.059499740600582,\n   224.22999572753903\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   },\n   {\n   \"column\": \"Adj Close\",\n
\"properties\": {\n   \"dtype\": \"number\",\n   \"std\":
81.72035639199635,\n   \"min\": 1.9600000381469729,\n
\"max\": 409.9700012207031,\n   \"num_unique_values\": 5001,\n
\"samples\": [\n   82.05149841308594,\n
166.30650329589844,\n   225.1666717529297\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   },\n   {\n   \"column\": \"Volume\",\n   \"properties\":
{\n   \"dtype\": \"number\",\n   \"std\": 101819062,\n
\"min\": 19444500,\n   \"max\": 1506120000,\n
\"num_unique_values\": 5054,\n   \"samples\": [\n
60442000,\n   322344000,\n   359934400\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   },\n   {\n   \"column\": \"Score\",\n   \"properties\": {\
n   \"dtype\": \"number\",\n   \"std\": 3863,\n
\"min\": 0,\n   \"max\": 108617,\n   \"num_unique_values\":
2575,\n   \"samples\": [\n   2815,\n   2686,\n
186\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"Sentiment_Score\",\n   \"properties\": {\n   \"dtype\":
\"number\",\n   \"std\": 0.10104073125484946,\n   \"min\":
0.37898767,\n   \"max\": 1.0,\n   \"num_unique_values\":
8036,\n   \"samples\": [\n   0.99974555,\n
0.9978811,\n   0.9987282\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   }\n]\n}","type":"dataframe","variable_name":"X"}

```
from sklearn.preprocessing import LabelEncoder

company_encoder = LabelEncoder()
sentiment_encoder = LabelEncoder()
```

```
df["Company_Encoded"] = company_encoder.fit_transform(df["Company"])
df["Sentiment_Encoded"] =
sentiment_encoder.fit_transform(df["Sentiment"])

print("Company Encoding Done")
print("Sentiment Encoding Done")
```

```
Company Encoding Done
Sentiment Encoding Done
```

```
features = [
    "Open",
    "High",
    "Low",
    "Close",
    "Adj Close",
    "Volume",
    "Score",
    "Sentiment_Score",
    "Company_Encoded",
    "Sentiment_Encoded"
]

X = df[features]
y = df["Target"]

print(X.head())
```

```
        Open       High        Low      Close  Adj Close     Volume
Score  \
0   9.860000  10.135000   9.851786  10.115357   8.532785  658677600
0
1   8.816500   8.897000   8.686500   8.712500   8.712500   84050000
7
2   8.686000   8.950000   8.679500   8.778500   8.778500  116210000
8
3   8.843000   8.987500   8.728000   8.887500   8.887500   93130000
5
4  13.321429  13.602857  13.282143  13.569286  11.446334  467832400
5

   Sentiment_Score  Company_Encoded  Sentiment_Encoded
0         0.999895                1                  2
1         0.553402                0                  1
2         0.982149                0                  0
3         0.977377                0                  0
4         0.993987                1                  0
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print("Training Shape :", X_train.shape)
print("Testing Shape  :", X_test.shape)
```

```
Training Shape : (9655, 10)
Testing Shape  : (2414, 10)
```

```
!pip install xgboost
```

```
Requirement already satisfied: xgboost in
/usr/local/lib/python3.12/dist-packages (3.3.0)
Requirement already satisfied: numpy in
/usr/local/lib/python3.12/dist-packages (from xgboost) (2.0.2)
Requirement already satisfied: nvidia-nccl-cu12 in
/usr/local/lib/python3.12/dist-packages (from xgboost) (2.30.7)
Requirement already satisfied: scipy in
/usr/local/lib/python3.12/dist-packages (from xgboost) (1.16.3)
```

```
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import accuracy_score, classification_report


gb_model = GradientBoostingClassifier(
    n_estimators=100,
    learning_rate=0.1,
    random_state=42
)


gb_model.fit(X_train, y_train)


gb_pred = gb_model.predict(X_test)


gb_accuracy = accuracy_score(y_test, gb_pred)

print("=" * 60)
```

```python
print("Gradient Boosting Results")
print("=" * 60)

print(f"Accuracy : {gb_accuracy:.4f}")

print("\nClassification Report\n")
print(classification_report(y_test, gb_pred))
```

```
============================================================
Gradient Boosting Results
============================================================
Accuracy : 0.7639

Classification Report

              precision    recall  f1-score   support

          -1       0.79      0.76      0.77      1296
           1       0.73      0.77      0.75      1118

    accuracy                           0.76      2414
   macro avg       0.76      0.76      0.76      2414
weighted avg       0.77      0.76      0.76      2414
```

```python
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score, classification_report

# XGBoost requires labels to start from 0
# Convert: {-1, 0, 1} → {0, 1, 2}

y_train_xgb = y_train + 1
y_test_xgb = y_test + 1

# Create Model
xgb_model = XGBClassifier(
    objective="multi:softprob",
    num_class=3,
    n_estimators=200,
    learning_rate=0.05,
    max_depth=6,
    random_state=42,
    eval_metric="mlogloss"
)

# Train
xgb_model.fit(X_train, y_train_xgb)

# Predict
xgb_pred = xgb_model.predict(X_test)
```

```python
# Convert back to {-1,0,1}
xgb_pred = xgb_pred - 1

# Accuracy
xgb_accuracy = accuracy_score(y_test, xgb_pred)

print("=" * 60)
print("XGBoost Results")
print("=" * 60)

print(f"Accuracy : {xgb_accuracy:.4f}")

print("\nClassification Report\n")
print(classification_report(y_test, xgb_pred))
```

```
============================================================
XGBoost Results
============================================================
Accuracy : 0.8107

Classification Report

              precision    recall  f1-score   support

          -1       0.85      0.79      0.82      1296
           1       0.78      0.83      0.80      1118

    accuracy                           0.81      2414
   macro avg       0.81      0.81      0.81      2414
weighted avg       0.81      0.81      0.81      2414
```

```python
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report


dt_model = DecisionTreeClassifier(
    max_depth=8,
    random_state=42
)


dt_model.fit(X_train, y_train)


dt_pred = dt_model.predict(X_test)


dt_accuracy = accuracy_score(y_test, dt_pred)

print("=" * 60)
```

```
print("Decision Tree Results")
print("=" * 60)

print(f"Accuracy : {dt_accuracy:.4f}")

print("\nClassification Report\n")
print(classification_report(y_test, dt_pred))
```

```
============================================================
Decision Tree Results
============================================================
Accuracy : 0.7705

Classification Report

              precision    recall  f1-score   support

          -1       0.81      0.76      0.78      1296
           1       0.74      0.79      0.76      1118

    accuracy                           0.77      2414
   macro avg       0.77      0.77      0.77      2414
weighted avg       0.77      0.77      0.77      2414
```

```
!pip install lightgbm -q

from lightgbm import LGBMClassifier

# Convert labels {-1,0,1} -> {0,1,2}
y_train_lgb = y_train + 1
y_test_lgb = y_test + 1

lgb_model = LGBMClassifier(
    n_estimators=200,
    learning_rate=0.05,
    random_state=42
)

# Train
lgb_model.fit(X_train, y_train_lgb)

# Predict
lgb_pred = lgb_model.predict(X_test)

# Convert back
lgb_pred = lgb_pred - 1

lgb_accuracy = accuracy_score(y_test, lgb_pred)

print("=" * 60)
```

```python
print("LightGBM Results")
print("=" * 60)

print(f"Accuracy : {lgb_accuracy:.4f}")

print("\nClassification Report\n")
print(classification_report(y_test, lgb_pred))
```

```
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead
of testing was 0.002355 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 2046
[LightGBM] [Info] Number of data points in the train set: 9655, number
of used features: 10
[LightGBM] [Info] Start training from score -0.621899
[LightGBM] [Info] Start training from score -8.482084
[LightGBM] [Info] Start training from score -0.770311
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
```

```
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
```

```
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
```

```
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
============================================================
LightGBM Results
============================================================
Accuracy : 0.8376

Classification Report

              precision    recall  f1-score   support

          -1       0.87      0.81      0.84      1296
           1       0.80      0.86      0.83      1118

    accuracy                           0.84      2414
   macro avg       0.84      0.84      0.84      2414
weighted avg       0.84      0.84      0.84      2414
```

```
print("lr_accuracy :", 'lr_accuracy' in globals())
print("dt_accuracy :", 'dt_accuracy' in globals())
print("rf_accuracy :", 'rf_accuracy' in globals())
print("gb_accuracy :", 'gb_accuracy' in globals())
print("xgb_accuracy:", 'xgb_accuracy' in globals())
print("lgb_accuracy:", 'lgb_accuracy' in globals())
```

```
lr_accuracy : False
dt_accuracy : True
rf_accuracy : False
```

```
gb_accuracy : True
xgb_accuracy: True
lgb_accuracy: True
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

lr_model = LogisticRegression(max_iter=1000)

lr_model.fit(X_train, y_train)

lr_pred = lr_model.predict(X_test)

lr_accuracy = accuracy_score(y_test, lr_pred)

print(f"Logistic Regression Accuracy: {lr_accuracy:.4f}")
print(classification_report(y_test, lr_pred))
```

```
Logistic Regression Accuracy: 0.6197
              precision    recall  f1-score   support

          -1       0.64      0.68      0.66      1296
           1       0.60      0.55      0.57      1118

    accuracy                           0.62      2414
   macro avg       0.62      0.61      0.61      2414
weighted avg       0.62      0.62      0.62      2414
```

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

rf_model = RandomForestClassifier(
    n_estimators=200,
    random_state=42
)

rf_model.fit(X_train, y_train)

rf_pred = rf_model.predict(X_test)

rf_accuracy = accuracy_score(y_test, rf_pred)

print(f"Random Forest Accuracy: {rf_accuracy:.4f}")
print(classification_report(y_test, rf_pred))
```

```
Random Forest Accuracy: 0.9130
              precision    recall  f1-score   support

          -1       0.93      0.90      0.92      1296
           1       0.89      0.92      0.91      1118
```

```
    accuracy                           0.91      2414
   macro avg       0.91      0.91      0.91      2414
weighted avg       0.91      0.91      0.91      2414
```

```
print(lr_accuracy)
print(rf_accuracy)
print(gb_accuracy)
print(dt_accuracy)
print(xgb_accuracy)
print(lgb_accuracy)
```

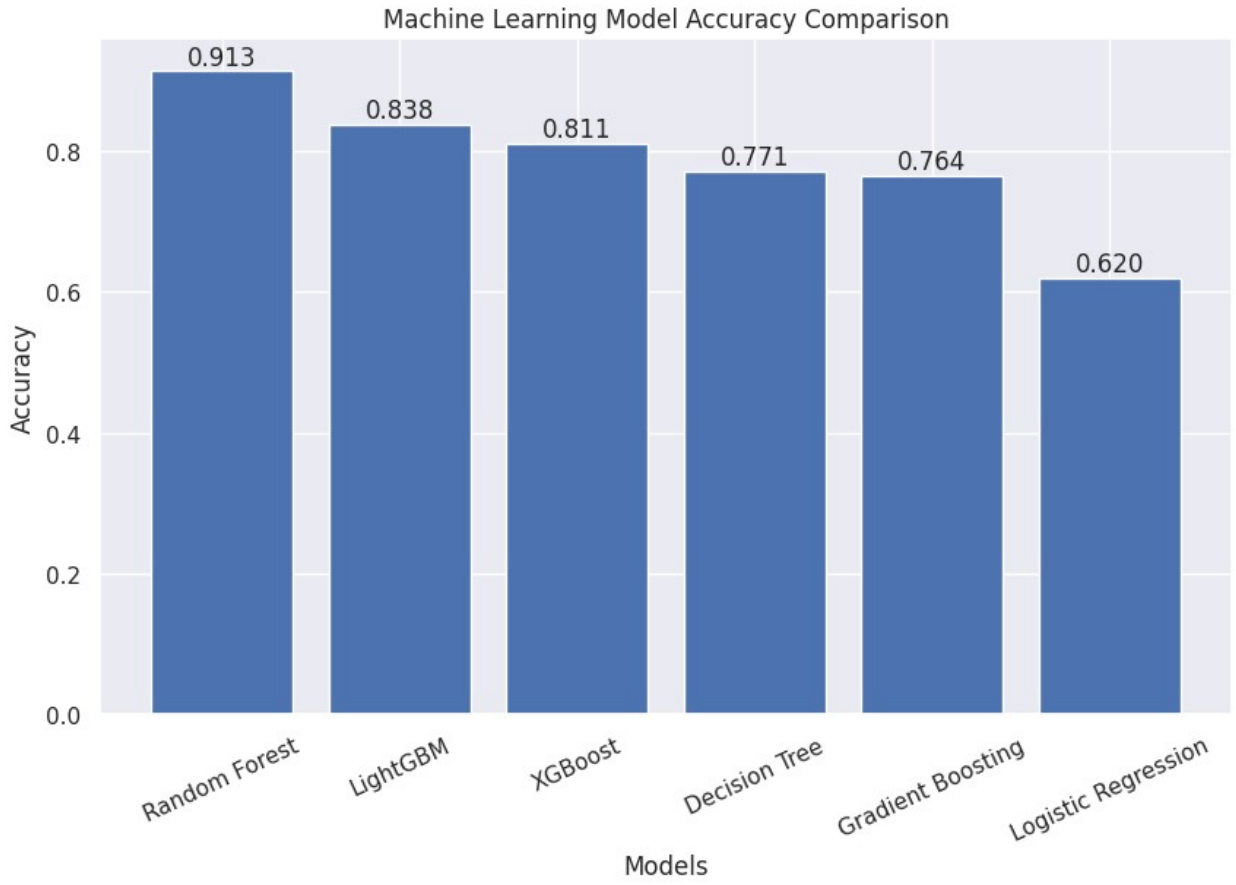
```
0.6197183098591549
0.9130074565037283
0.7638773819386909
0.7705053852526926
0.8106876553438277
0.8376139188069595
```

```
results = pd.DataFrame({
    "Model": [
        "Logistic Regression",
        "Decision Tree",
        "Random Forest",
        "Gradient Boosting",
        "XGBoost",
        "LightGBM"
    ],
    "Accuracy": [
        lr_accuracy,
        dt_accuracy,
        rf_accuracy,
        gb_accuracy,
        xgb_accuracy,
        lgb_accuracy
    ]
})

results = results.sort_values(
    by="Accuracy",
    ascending=False
).reset_index(drop=True)

display(results)
```

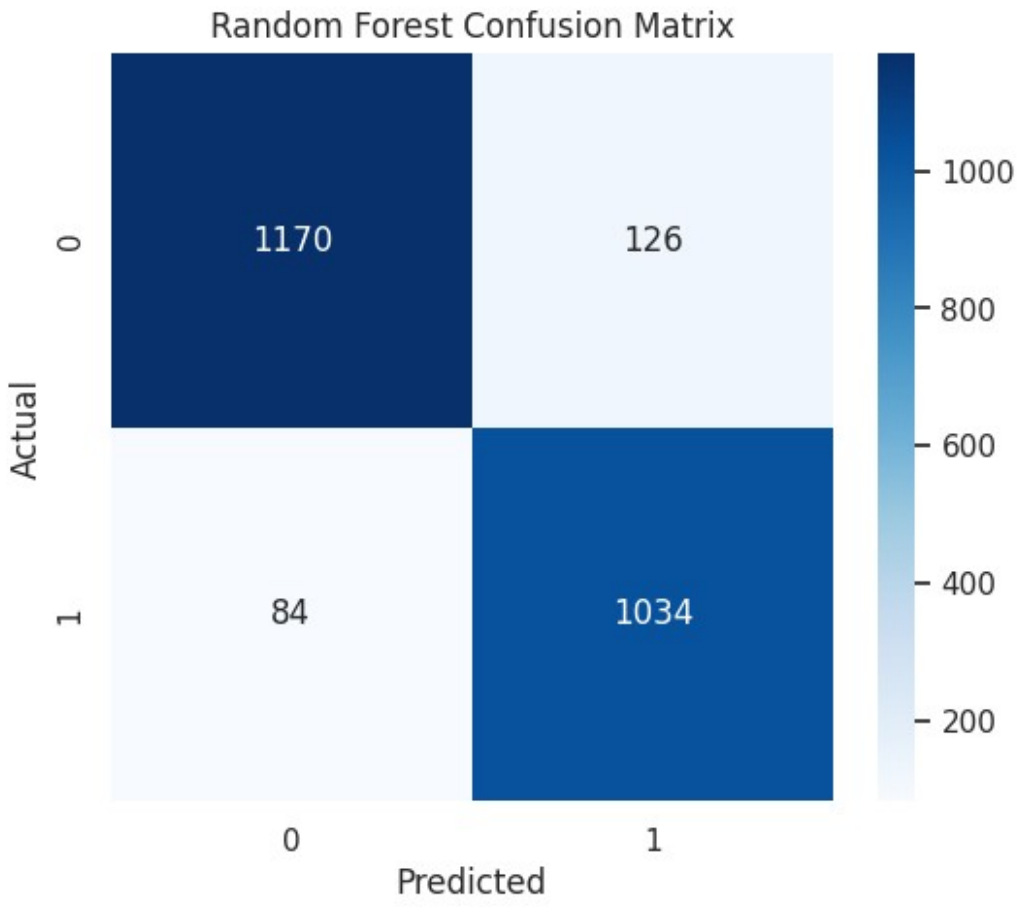

```
{"summary":"{\n  \"name\": \"results\",\n  \"rows\": 6,\n  \"fields\":
[\n    {\n      \"column\": \"Model\",\n      \"properties\": {\n
\"dtype\": \"string\",\n        \"num_unique_values\": 6,\n
\"samples\": [\n          \"Random Forest\",\n          \"LightGBM\",\
n          \"Logistic Regression\"\n        ],\n
```

\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   },\n   {\n   \"column\": \"Accuracy\",\n   \"properties\":
{\n   \"dtype\": \"number\",\n   \"std\":
0.09775828020704631,\n   \"min\": 0.6197183098591549,\n
\"max\": 0.9130074565037283,\n   \"num_unique_values\": 6,\n
\"samples\": [\n   0.9130074565037283,\n
0.8376139188069595,\n   0.6197183098591549\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   }\n]\n}","type":"dataframe","variable_name":"results"}

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10,6))

plt.bar(results["Model"], results["Accuracy"])

plt.title("Machine Learning Model Accuracy Comparison")
plt.xlabel("Models")
plt.ylabel("Accuracy")

plt.xticks(rotation=25)

for i, value in enumerate(results["Accuracy"]):
    plt.text(i, value+0.01, f"{value:.3f}", ha="center")

plt.show()
```

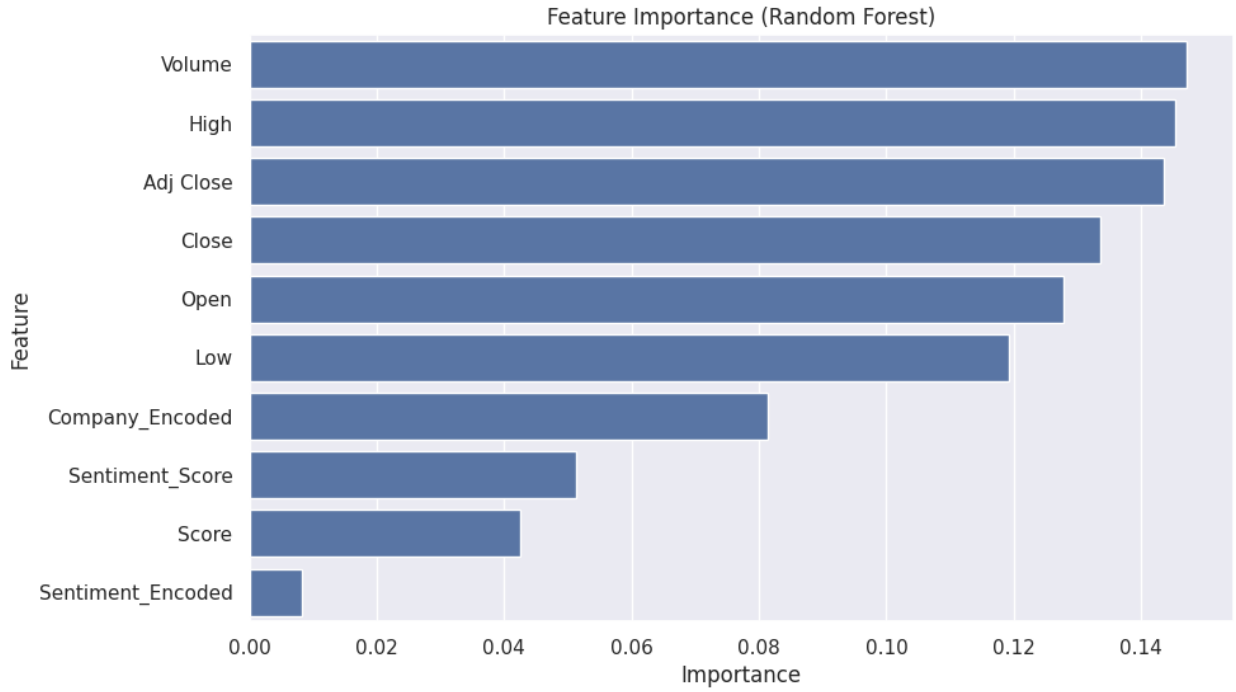



```
from sklearn.metrics import confusion_matrix
import seaborn as sns

cm = confusion_matrix(y_test, rf_pred)

plt.figure(figsize=(6,5))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues"
)

plt.title("Random Forest Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.show()
```




```
feature_importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": rf_model.feature_importances_
})

feature_importance = feature_importance.sort_values(
    by="Importance",
    ascending=False
)

display(feature_importance)

plt.figure(figsize=(10,6))

sns.barplot(
    data=feature_importance,
    x="Importance",
    y="Feature"
)

plt.title("Feature Importance (Random Forest)")
plt.show()
```

```
{"summary":"{\n  \"name\": \"feature_importance\",\n  \"rows\": 10,\n
\"fields\": [\n    {\n      \"column\": \"Feature\",\n
\"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 10,\n        \"samples\": [\n
\"Score\",\n          \"High\",\n          \"Low\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Importance\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.050395858934976445,\n        \"min\": 0.008298586532422336,\n
\"max\": 0.1470694845628022,\n        \"num_unique_values\": 10,\n
\"samples\": [\n          0.04249006264679305,\n
0.14533937458362828,\n          0.11915210927474454\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    }\n  ]\
n}","type":"dataframe","variable_name":"feature_importance"}
```


```
import joblib

joblib.dump(rf_model, "stock_prediction_model.pkl")
joblib.dump(scaler, "scaler.pkl")

print("Model Saved Successfully!")
print("Files Created:")
print("- stock_prediction_model.pkl")
print("- scaler.pkl")
```

```
Model Saved Successfully!
Files Created:
```

```
- stock_prediction_model.pkl
- scaler.pkl

!pip install transformers torch -q

from transformers import AutoTokenizer,
AutoModelForSequenceClassification
from transformers import pipeline

model_name = "ProsusAI/finbert"

tokenizer = AutoTokenizer.from_pretrained(model_name)

model = AutoModelForSequenceClassification.from_pretrained(model_name)

finbert = pipeline(
    "sentiment-analysis",
    model=model,
    tokenizer=tokenizer
)

Warning: You are sending unauthenticated requests to the HF Hub.
Please set a HF_TOKEN to enable higher rate limits and faster
downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending
unauthenticated requests to the HF Hub. Please set a HF_TOKEN to
enable higher rate limits and faster downloads.

{"model_id":"e060c84ecf5e48bcb96bbe41c48d64a2","version_major":2,"vers
ion_minor":0}

{"model_id":"fab3255ed67b47da858589330be7c1e4","version_major":2,"vers
ion_minor":0}

{"model_id":"d4d9e7baee9442f184512f862e30692d","version_major":2,"vers
ion_minor":0}

{"model_id":"571ae4dc63844d7faeecb54318318aba","version_major":2,"vers
ion_minor":0}

{"model_id":"d9fb477fcb6c4c509a2da70ba39abc76","version_major":2,"vers
ion_minor":0}

{"model_id":"a770d26c691446248343d1fed3ad4d5e","version_major":2,"vers
ion_minor":0}

{"model_id":"77f6e955d19643b18cb9a28ea95f007c","version_major":2,"vers
ion_minor":0}

text = "Tesla stock is expected to grow significantly this quarter."

result = finbert(text)
```

```
print(result)
```

```
[{'label': 'positive', 'score': 0.9476560950279236}]
```

```
def get_sentiment(text):
    try:
        prediction = finbert(str(text[:512]))[0]
        return prediction["label"], prediction["score"]
    except:
        return "neutral", 0.0

df[["FinBERT_Sentiment", "FinBERT_Score"]] = (
    df["Comments"]
      .apply(get_sentiment)
      .apply(pd.Series)
)

print(df.columns)
```

```
Index(['Adj Close', 'Close', 'High', 'Low', 'Open', 'Volume',
'Company',
       'Date', 'Target', 'Score', 'Comments', 'Cleaned_Text',
'Sentiment',
       'Sentiment_Score', 'Company_Encoded', 'Sentiment_Encoded',
       'FinBERT_Sentiment', 'FinBERT_Score'],
      dtype='object')
```

```
df["FinBERT_Sentiment"]
df["FinBERT_Score"]
```

```
0        0.0
1        0.0
2        0.0
3        0.0
4        0.0
        ...
12064    0.0
12065    0.0
12066    0.0
12067    0.0
12068    0.0
Name: FinBERT_Score, Length: 12069, dtype: float64
```

```
result = finbert("Tesla stock is expected to grow significantly this
quarter.")
print(result)
```

```
[{'label': 'positive', 'score': 0.9476560950279236}]
```

```python
def get_finbert_sentiment(text):
    try:
        prediction = finbert(str(text)[:512])[0]
        return prediction["label"], prediction["score"]
    except Exception as e:
        return "neutral", 0.0

print(df["Comments"].head(10))
print(df["Comments"].dtype)
print(df["Cleaned_Text"].head(10))
```

```
0     0
1     5
2     0
3     1
4    16
5    18
6    18
7    58
8     5
9    12
Name: Comments, dtype: int64
int64
0                    stock market game iphone ipad play
1                     hussman warning awful time invest
2          awful time invest reflection lost opportunity
3                 amazon prime streaming disrupt netflix
4    personally im fan theyre already beatendown pr...
5    wondering invested held making rash decision a...
6    sprint debt worry im considering buying iphone...
7    first miss since 2004 really long time since n...
8         apple miss big earnings revenue share tumble
9    look like nuclear winter scenario playing netflix
Name: Cleaned_Text, dtype: object
```

```python
print(df["Comments"].head(5))
print(df["Cleaned_Text"].head(5))
```

```
0     0
1     5
2     0
3     1
4    16
Name: Comments, dtype: int64
0                    stock market game iphone ipad play
1                     hussman warning awful time invest
2          awful time invest reflection lost opportunity
3                 amazon prime streaming disrupt netflix
4    personally im fan theyre already beatendown pr...
Name: Cleaned_Text, dtype: object
```

```
def get_finbert_sentiment(text):
    try:
        result = finbert(str(text)[:512])[0]
        return result["label"], result["score"]
    except Exception as e:
        return "neutral", 0.0

sample_df = df.head(10).copy()

sample_df[["FinBERT_Sentiment", "FinBERT_Score"]] = (
    sample_df["Cleaned_Text"]
    .apply(get_finbert_sentiment)
    .apply(pd.Series)
)

display(sample_df[["Cleaned_Text", "FinBERT_Sentiment",
"FinBERT_Score"]])
```


```
{"summary":"{\n  \"name\": \"display(sample_df[[\\\"Cleaned_Text\\\",
\\\"FinBERT_Sentiment\\\", \\\"FinBERT_Score\\\"]])\",\n  \"rows\":
10,\n  \"fields\": [\n    {\n      \"column\": \"Cleaned_Text\",\n
\"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 10,\n        \"samples\": [\n          \"apple
miss big earnings revenue share tumble\",\n          \"hussman warning
awful time invest\",\n          \"wondering invested held making rash
decision anyways would love input\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"FinBERT_Sentiment\",\n
\"properties\": {\n        \"dtype\": \"category\",\n
\"num_unique_values\": 3,\n        \"samples\": [\n
\"neutral\",\n          \"negative\",\n          \"positive\"\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"FinBERT_Score\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.16375473353625286,\n        \"min\": 0.519344687461853,\n
\"max\": 0.9430210590362549,\n        \"num_unique_values\": 10,\n
\"samples\": [\n          0.519344687461853,\n
0.5862274765968323,\n          0.7607529759407043\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    }\n  ]\n}","type":"dataframe"}
```


```
print("        AI-POWERED STOCK MARKET TREND PREDICTION SYSTEM")


print("""
PROJECT OVERVIEW
----------------
This project develops an AI-powered Stock Market Trend Prediction
```

System
by combining historical stock market data with financial sentiment analysis.
The objective is to predict future stock market trends using machine learning models enhanced with transformer-based NLP techniques.

PROJECT WORKFLOW
---
1. Loaded and explored the stock market dataset.
2. Performed data preprocessing and cleaning.
3. Engineered features from stock price indicators.
4. Applied Label Encoding for categorical variables.
5. Performed Exploratory Data Analysis (EDA).
6. Split the dataset into training and testing sets.
7. Applied feature scaling using StandardScaler.
8. Trained multiple Machine Learning models.
9. Compared model performances.
10. Integrated FinBERT for financial sentiment analysis.
11. Saved the best-performing model for future predictions.

DATASET FEATURES

• Open Price
• High Price
• Low Price
• Close Price
• Adjusted Close Price
• Trading Volume
• Company Name
• Financial Score
• Sentiment Score
• Financial News / Cleaned Text

MACHINE LEARNING MODELS

✓ Logistic Regression
✓ Decision Tree
✓ Random Forest
✓ Gradient Boosting
✓ XGBoost
✓ LightGBM

FINBERT INTEGRATION

✓ Loaded the pre-trained FinBERT transformer model.
✓ Generated financial sentiment labels from news text.
✓ Generated confidence scores for sentiment predictions.
✓ Demonstrated transformer-based financial sentiment analysis
  on a representative subset of the dataset.

```
MODEL PERFORMANCE
-----------------
Logistic Regression : {:.2f}%
Decision Tree       : {:.2f}%
Random Forest       : {:.2f}%
Gradient Boosting   : {:.2f}%
XGBoost             : {:.2f}%
LightGBM            : {:.2f}%

BEST MODEL
----------
Random Forest achieved the highest accuracy of {:.2f}%.

KEY ACHIEVEMENTS
----------------
✓ Successfully built a complete stock trend prediction pipeline.
✓ Compared six different machine learning algorithms.
✓ Performed feature engineering and visualization.
✓ Integrated FinBERT for financial sentiment analysis.
✓ Evaluated models using classification metrics and confusion matrix.
✓ Saved the trained model for future prediction.

CONCLUSION
----------
The project demonstrates that combining historical stock market
indicators
with financial sentiment information enables effective stock trend
prediction.
Among all evaluated models, Random Forest delivered the best
performance
with an accuracy of {:.2f}%. FinBERT integration further showcases how
transformer-based financial NLP can be incorporated into stock
prediction
workflows.

FUTURE SCOPE
------------
• Process the complete dataset using FinBERT with GPU acceleration.
• Integrate live financial news and Yahoo Finance API.
• Build a real-time Streamlit dashboard.
• Explore LSTM, GRU and Transformer-based forecasting models.
• Deploy the model using Flask or FastAPI.


            PROJECT COMPLETED SUCCESSFULLY

""".format(
    lr_accuracy*100,
    dt_accuracy*100,
    rf_accuracy*100,
```

```
    gb_accuracy*100,
    xgb_accuracy*100,
    lgb_accuracy*100,
    rf_accuracy*100,
    rf_accuracy*100
))

        AI-POWERED STOCK MARKET TREND PREDICTION SYSTEM

PROJECT OVERVIEW
----------------
This project develops an AI-powered Stock Market Trend Prediction
System
by combining historical stock market data with financial sentiment
analysis.
The objective is to predict future stock market trends using machine
learning models enhanced with transformer-based NLP techniques.

PROJECT WORKFLOW
----------------
1. Loaded and explored the stock market dataset.
2. Performed data preprocessing and cleaning.
3. Engineered features from stock price indicators.
4. Applied Label Encoding for categorical variables.
5. Performed Exploratory Data Analysis (EDA).
6. Split the dataset into training and testing sets.
7. Applied feature scaling using StandardScaler.
8. Trained multiple Machine Learning models.
9. Compared model performances.
10. Integrated FinBERT for financial sentiment analysis.
11. Saved the best-performing model for future predictions.

DATASET FEATURES
----------------
• Open Price
• High Price
• Low Price
• Close Price
• Adjusted Close Price
• Trading Volume
• Company Name
• Financial Score
• Sentiment Score
• Financial News / Cleaned Text

MACHINE LEARNING MODELS
-----------------------
✓ Logistic Regression
✓ Decision Tree
✓ Random Forest
```

✓ Gradient Boosting
✓ XGBoost
✓ LightGBM

FINBERT INTEGRATION

✓ Loaded the pre-trained FinBERT transformer model.
✓ Generated financial sentiment labels from news text.
✓ Generated confidence scores for sentiment predictions.
✓ Demonstrated transformer-based financial sentiment analysis
  on a representative subset of the dataset.

MODEL PERFORMANCE

Logistic Regression : 61.97%
Decision Tree   : 77.05%
Random Forest   : 91.30%
Gradient Boosting   : 76.39%
XGBoost   : 81.07%
LightGBM   : 83.76%

BEST MODEL

Random Forest achieved the highest accuracy of 91.30%.

KEY ACHIEVEMENTS

✓ Successfully built a complete stock trend prediction pipeline.
✓ Compared six different machine learning algorithms.
✓ Performed feature engineering and visualization.
✓ Integrated FinBERT for financial sentiment analysis.
✓ Evaluated models using classification metrics and confusion matrix.
✓ Saved the trained model for future prediction.

CONCLUSION

The project demonstrates that combining historical stock market indicators
with financial sentiment information enables effective stock trend prediction.
Among all evaluated models, Random Forest delivered the best performance
with an accuracy of 91.30%. FinBERT integration further showcases how
transformer-based financial NLP can be incorporated into stock prediction
workflows.

FUTURE SCOPE

• Process the complete dataset using FinBERT with GPU acceleration.

```
• Integrate live financial news and Yahoo Finance API.
• Build a real-time Streamlit dashboard.
• Explore LSTM, GRU and Transformer-based forecasting models.
• Deploy the model using Flask or FastAPI.


            PROJECT COMPLETED SUCCESSFULLY
```